

全系统模拟器 MMU 不命中路径自适应优化

Adaptive Optimization of MMU Miss Handling in Full-System Emulators

摘要

针对全系统动态二进制翻译器中内存管理单元（MMU）不命中处理开销较高的问题，本文提出大页转换缓存、非叶页表缓存及自适应收益门控。新增结构仅在主软件转换后备缓冲器和牺牲缓冲器均未命中后启用，不增加命中路径指令。固定 4096 项主缓冲器并关闭牺牲缓冲器时，大页缓存和页表缓存在微基准上的加速比分别为 1.397 和 1.065。在 7 个固定工作量应用中，大页缓存和组合方案的几何平均加速比分别为 1.142 和 1.135，嵌套虚拟化内层负载的最高加速比为 2.304，429.mcf 的组合方案加速比为 1.137。自适应组合方案在固定容量 429.mcf 上取得 1.212 倍加速，并在无大页复用的负对照中旁路 85.7% 的候选查询。实验表明，所提方法能够利用地址转换覆盖后的剩余局部性；在默认软件配置和部分应用中尚未获得加速，仍需结合负载特征及硬件辅助异构虚拟化平台进一步研究。

关键词：全系统模拟；动态二进制翻译；软件转换后备缓冲器；大页；页表遍历；自适应缓存；地址转换

Abstract

Memory-management-unit (MMU) miss handling remains costly in full-system dynamic binary translators. This paper presents a large-page translation cache, a non-leaf page-table cache, and an adaptive benefit controller. The new structures are accessed only after both the primary software translation lookaside buffer and its victim buffer miss, leaving the common hit path unchanged. With a fixed 4096-entry primary buffer and no victim buffer, the large-page and page-table caches achieve speedups of 1.397 and 1.065 on their respective microbenchmarks. Across seven fixed-work applications, the geometric-mean speedups of the large-page and combined schemes are 1.142 and 1.135, and the inner nested-virtualization workload reaches 2.304. The adaptive combination achieves 1.212 on 429.mcf and bypasses 85.7% of candidate queries in a negative control without large-page reuse. Some applications and the default software configuration do not yet benefit, motivating further workload-guided optimization and evaluation on heterogeneous virtualization platforms with hardware TLB acceleration.

Keywords: full-system emulation; dynamic binary translation; software TLB; large page; page-table walk; adaptive caching; address translation

1 引言

全系统动态二进制翻译需要模拟客户机处理器、内存管理单元和设备，并将每次客户机访存转换为安全的宿主访问。QEMU[4]在生成代码中内联软件转换后备缓冲器（software translation lookaside buffer, SoftTLB）查询，使常见命中只执行索引、标签比较和地址修正；未命中则进入目标体系结构页表遍历、地址空间解析和表项安装过程。Tong 等在早期 QEMU 上测得访存模拟平均占执行时间的 38.1%，并通过容量、关联度、大页和填充优化获得 24.4% 的平均加速[1]。此后 QEMU 引入动态伸缩主表和全相联牺牲 SoftTLB，早期瓶颈不能直接外推。近年的 Victima、Utopia 等工作利用缓存资源或受限映射扩大硬件地址转换覆盖范围[8-9]，Ninja 针对嵌套翻译减少多阶段遍历[10]，相关工作负载研究也表明地址转换开销随访问模式和并发规模显著变化[11]。这些机制主要面向硬件。面向跨指令集体系结构模拟，宿主页表辅助方案（HSPT）借助宿主页表和 TLB，

基于双 TLB 的内存虚拟化方案 (BTMMU) 以硬件异常缩短公共转换路径[12-13]; 影子映射缺失后仍需由软件遍历客户机页表并建立映射。对纯软件模拟器而言, 新增查询若进入每次访存, 会增加宿主指令和缓存压力。为此, 本文只优化主 SoftTLB 和牺牲 TLB 均未命中的 MMU 不命中路径, 设计大页转换缓存和非叶页表缓存, 再按运行期复用收益自适应旁路低效查询。实验同时覆盖 QEMU 默认动态 SoftTLB 和固定容量压力配置: 前者用于判断现代软件机制后的剩余空间, 后者作为有限硬件转换覆盖的受控代理。结果显示, 定向微基准和固定容量的 429.mcf 存在明确收益, 默认动态配置则基本持平, 自适应门控只能限制而不能保证消除回退。本文的贡献包括: (1) 在现代 QEMU 的 MMU 不命中路径实现大页转换复用和非叶页表复用, 并区分嵌套翻译中的失效覆盖范围与线性重建范围; (2) 设计无需硬件计数器的分窗自适应门控, 用可避免的 TLB 表项填充和页表访问识别低收益区间, 同时保持生成代码命中路径不变; (3) 建立计时、性能采样、机制计数与客户机访存分母相分离的验证方法, 在固定容量代理、QEMU 默认动态容量及无复用负对照中给出收益和边界, 并讨论其对 HSPT、BTMMU 等硬件辅助跨体系结构模拟剩余 MMU 不命中路径的适用性。

2 访存通路 with 开销测量

2.1 术语与方案定义

本文所称客户机是由 QEMU 模拟的目标计算机, 宿主机是实际执行 QEMU 的计算机; 动态二进制翻译是将客户机指令在运行时转换为宿主机指令的执行方法。内存管理单元 (memory management unit, MMU) 负责将虚拟地址转换为物理地址。转换后备缓冲器 (translation lookaside buffer, TLB) 缓存近期地址转换结果; 查询到有效表项称为 TLB 命中, 未查询到有效表项称为 TLB 未命中。QEMU 以软件数据结构实现的 TLB 称为 SoftTLB, 其中直接服务生成代码查询的表项称为主 SoftTLB, 保存近期被主表替换表项的小型全相联结构称为牺牲 TLB (victim TLB)。页表遍历 (page-table walk) 是按照客户机页表层级读取页表项并完成地址转换的过程; TLB 表项填充是将转换结果写入 SoftTLB、供客户机指令重新执行的过程。

本文将“MMU 不命中路径”明确定义为: 一次客户机访存连续未命中主 SoftTLB 和牺牲 TLB 后, 从目标体系结构地址转换、页表遍历和内存区域解析开始, 直至新表项写入主 SoftTLB 的处理路径。该定义不包括生成代码中的主 SoftTLB 命中查询。2 MiB 大页是相对于 4 KiB 普通页具有更大映射粒度的页; 透明大页 (transparent huge page, THP) 是 Linux 在不改变应用程序接口的条件下自动建立大页映射的机制。

实验方案统一定义如下: 基线方案 (Baseline, Base) 关闭本文新增的缓存; 大页缓存方案 (large-page translation cache, LP) 仅启用大页转换缓存; 页表缓存方案 (page-table-walk cache, PTW) 仅启用非叶页表缓存; 组合方案 (LP+PTW) 同时启用 LP 和 PTW; 自适应组合方案 (Adaptive LP+PTW, A-LP+PTW) 同时启用两种缓存及收益门控。后文表格仅使用上述缩写, 所有加速比均定义为 Base 平均执行时间除以候选方案平均执行时间, 大于 1 表示候选方案缩短了执行时间。

2.2 QEMU 系统模式访存通路

QEMU 的翻译代码块在执行客户机读写指令时首先查询当前内存管理模式对应的 SoftTLB。命中项包含标签、权限和宿主地址修正量, 可直接形成宿主访问。主表未命中时, 运行时函数先查询 8 项全相联牺牲 TLB; 再次未命中后调用目标体系结构 TLB 表项填充函数。对 x86-64 客户机, 填充函数可能执行四级或五级页表遍历, 也可能在嵌套虚拟化中继续遍历第二阶段页表。得到客户机物理地址后, QEMU 根据内存区域描述解析访问目标, 并由 SoftTLB 表项安装函数写入普通页表项。图 1 给出通路及本文缓存与门控的位置。

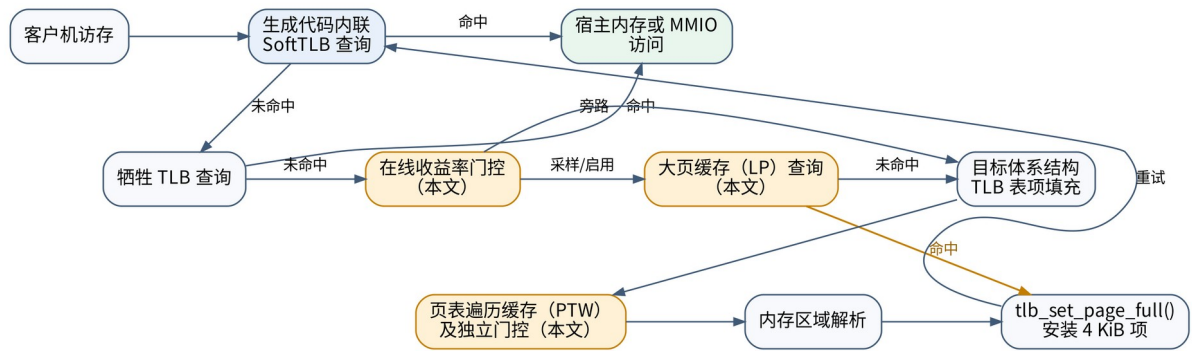


图 1 QEMU SoftTLB 访存通路及优化位置

大多数访存停留在图 1 上方的内联命中路径。若在该路径增加一次哈希、分支或额外访存，其成本会被所有客户机内存操作放大。本文因而把新增结构放在两级 SoftTLB 均未命中之后：LP 位于目标体系结构 TLB 表项填充之前，PTW 位于 x86 页表遍历函数内部。

2.3 分离式测量方法

本文分别进行运行时间测量、函数级性能采样、机制事件计数和客户机访存次数统计。运行时间测量使用不含机制计数的 QEMU，并关闭即时翻译代码性能映射文件，以避免额外磁盘写入。函数级性能采样使用 Linux perf 工具；机制计数版本记录主表未命中、牺牲 TLB 命中、TLB 表项填充、页表遍历和缓存事件；微型代码生成器（Tiny Code Generator，TCG）插件在客户机标记区间内统计读写指令次数。带性能采样或机制计数的运行不参与加速比计算。

对除原子特殊路径外的访问，事件计数应满足

$$N_f = N_p - N_v$$

其中， N_f 为 TLB 表项填充次数， N_p 为主 SoftTLB 未命中次数， N_v 为牺牲 TLB 命中次数。

分析器在调用来源、读写类型和内存管理模式三个维度检查该关系，并验证客户机标记、远端返回码、二进制哈希和运行参数。全部计数记录中实际出现的 134 个“调用来源—访问类型—内存管理模式”组合，其计数残差均为 0。

2.4 MMU 不命中路径开销

表 1 给出六个应用负载的 SoftTLB 事件率与宿主周期占比。“软件 MMU 相关 C 函数”是指性能采样结果中能够归因到 QEMU 软件 MMU 相关 C 函数的周期，不包含生成代码内联的 SoftTLB 命中查询；事件率来自相互独立的客户机访存次数统计和机制计数运行。

表 1 QEMU 软件 MMU 不命中路径开销

工作负载	软件 MMU 相关 C 函数周期占比	数据访问 TLB 填充次数/客户机访存次数	牺牲 TLB 命中占主表未命中的比例
sysbench 随机 4 KiB 写访问	5.21%	0.0587%	44.46%
stress-ng TLB 失效压力	16.84%	0.4948%	19.14%
DaCapo avrora	6.61%	0.0372%	83.15%
嵌套 QEMU/KVM 随机访存	13.87%	0.1276%	85.13%
SPEC CPU2006 429.mcf 训练输入	15.22%	0.2649%	8.95%
GAPBS PageRank (2 ²⁰ 个顶点)	1.17%	0.0246%	88.00%

六个负载的数据访问 TLB 填充次数均低于客户机访存次数的 0.5%，但软件 MMU 相关 C 函数最高占宿主周期的 16.84%，说明 MMU 不命中路径具有低频、高单次成本特征。429.mcf 中 91.05% 的数据访问主 SoftTLB 未命中最终进入表项填充，平均每次页表遍历访问 4.19 层，是最明显的应用级地址转换压力负载；GAPBS 中 88.00% 的主表未命中被牺牲 TLB 处理，稳定阶段的优化空间较小。

为排除动态扩容和牺牲表的遮蔽效应，本文进一步将每个内存管理模式的主表固定为 4096 项并关闭牺牲 TLB。在该受控配置下，六个负载的软件 MMU 相关 C 函数平均占比为 13.39%，429.mcf 最高为 24.18%。这一数值仍低于文献[1]的完整访存模拟占比，主要原因是内联命中查询无法从客户机有效计算中单独归因，因此 13.39% 是本文定义的 MMU 不命中路径开销下界。

2.5 硬件辅助跨体系结构模拟中的 MMU 不命中路径

宿主页表辅助地址转换方案（HSPT）将客户机地址空间映射到宿主页表，使常见访问利用宿主 MMU 和 TLB 完成转换；影子映射缺失时，由信号处理程序遍历客户机页表并建立映射[12]。基于双 TLB 的内存虚拟化方案（BTMMU）则在宿主 MMU 中增加客户机 TLB 和宿主机 TLB，先从内核影子页表重填客户机 TLB，影子项缺失后再进入非特权异常处理程序[13]。相关对比实验表明，QEMU 的 SoftTLB 未命中、HSPT 的段错误异常和 BTMMU 的客户机 TLB 无效异常均需要遍历客户机页表并建立新映射[13]。

因此，硬件加速能够显著降低公共命中路径和异常率，却没有消除剩余映射构造成本。本文以动态主表和 8 项牺牲 TLB 表示现代 QEMU 的软件默认机制，以固定 4096 项主表并关闭牺牲 TLB 构造容量压力。后者仅是有限硬件转换缓存的受控近似，不复现具体硬件的关联度、替换策略和异常入口延迟；其结果用于评估 MMU 不命中路径优化的潜在空间，而非直接预测 HSPT 或 BTMMU 的整机加速比。

3 MMU 不命中路径缓存设计

3.1 设计约束

两个缓存遵循相同约束：不改变 QEMU 的完整 SoftTLB 表项结构，不在生成代码的公共命中路径增加指令；缓存命中后仍调用原有表项安装函数，不绕过随机存取存储器（RAM）、内存映射输入输出（MMIO）、只读存储、脏页和观察点处理。缓存采用统一的代际计数器完成失效：失效操作递增计数器，旧代际表项随后不能命中。运行时提供关闭、启用、探测和自适应四种模式。探测模式执行缓存查询但仍返回原处理路径，用于测量查询成本和潜在覆盖率；自适应模式依据运行期复用率启用或旁路缓存。

3.2 大页转换缓存

QEMU 即使识别到 2 MiB 客户机大页，也通常按 4 KiB 粒度向主 SoftTLB 安装表项。随机访问同一大页的不同子页会重复进入 TLB 表项填充和页表遍历。本文为每个内存管理模式设置 32 组、4 路，共 128 项的大页转换缓存。缓存项保存客户机虚拟基址、物理基址、完整转换元数据、权限、线性转换范围和代际编号。只有目标体系结构明确给出大于 4 KiB 的线性转换范围时才允许插入。

设虚拟地址为 v ，缓存基址为 v_b 和 p_b ，线性范围为 2^S 。当 v 位于该范围内时，物理地址按式（1）重建：

$$p = p_b + (v - v_b), 0 \leq v - v_b < 2^S \quad (1)$$

命中后使用保存的完整转换元数据安装当前 4 KiB 子页。带写入失效标志的表项不服务写访问，权限不足时也不命中。该设计减少目标体系结构 TLB 表项填充和页表遍历，但仍保留主表查询、4 KiB 表项安装及原有地址空间处理。

实现中必须区分“失效覆盖页大小”和“可线性重建的转换范围”。嵌套翻译为保证失效正确性可能取两阶段页大小的较大值，而线性映射只能取两阶段范围的较小值。本文新增 `lg_translation_size` 字段表达后者，避免以过大范围重建相邻子页的错误物理地址。

3.3 非叶页表缓存

非叶页表缓存为每个虚拟处理器设置 64 组、4 路，保存 x86-64 第 2 级至第 4 级页表遍历的中间结果。缓存索引键由页表根地址寄存器 CR3、分页模式、MMU 索引、页表遍历索引、层级及虚拟地址前缀组成，缓存值包含下一层页表物理地址和累计权限。命中后页表遍历从更低层继续，叶页表项仍重新读取，以保留最终映射及访问位、脏位语义。该实现覆盖四级页表、五级页表和嵌套页表路径，并在全局、单页和地址范围 TLB 失效时递增代际计数器。

与硬件 MMU 缓存不同，软件查询本身包含散列计算、标签比较和分支。只有被跳过页表访问的宿主成本高于查询成本时，非叶缓存才能形成净收益。因此该缓存不进入公共 SoftTLB 命中路径，并与大页缓存独立启停。

3.4 自适应低收益旁路

MMU 不命中路径缓存仍会为每次候选事件增加标签查询和分支。缓存命中率不能精确预测端到端加速，但在复用不足时继续查询只会增加成本。本文因而为每个虚拟处理器的两类缓存设置独立的三阶段控制器。控制器先观察 2048 次候选查询；若达到收益阈值，则连续启用 16384 次，否则旁路随后 16384 次候选事件，再重新观察。查询、插入和控制器均位于主 SoftTLB 与牺牲 TLB 未命中之后，不进入生成代码的公共命中路径。

对 LP，一次命中可避免一次目标体系结构 TLB 表项填充，窗口内至少每 32 次查询命中 1 次即视为有效。对 PTW，命中层级 1 可跳过 5-1 个上层访问，因而按跳过层数加权，要求平均每 8 次查询至少跳过 1 层。该策略使用软件可直接获得的机制事件作为低成本近似指标，不需要宿主性能计数器，也不假定命中与整机收益严格成正比。固定阈值用于识别明显无复用区间；接近盈亏边界的情况仍须由离线计时判断。

3.5 正确性验证

测试覆盖 4 KiB 与透明大页、随机与连续访问、读写权限、内存权限修改系统调用 `mprotect`、页级和范围失效、嵌套翻译及校验和一致性。`mprotect` 测试完成 32768 次访问并得到相同校验和 4177920，LP 命中 29893 次且经历 27 次失效；PTW 经历 16 次失效。嵌套虚拟化负载的 Base、LP 和 PTW 运行均得到校验和 66846720，64 MiB 映射中有 63488 KiB 经 Linux 内存映射信息确认为透明大页。上述 `lg_translation_size` 边界通过嵌套大页回归用例验证。

自适应模式下，最终 `mprotect` 回归执行 1048576 次访问，校验和为 133693440；LP 和 PTW 分别经历 30 次和 64 次失效，TLB 表项填充计数关系保持为 0 残差。该结果通过页表遍历、容量快照和 QEMU 版本校验。

4 实验方法

4.1 实验环境

表 2 给出实验环境。性能评价使用不含机制计数的同一 QEMU 可执行文件，客户机磁盘采用写时复制格式的临时快照。各方案在每轮内按固定种子随机排序，以减弱温度和后台负载的时间趋

势。正式测试将 QEMU 绑定到性能核（P-Core）的逻辑处理器 7，并将操作系统调度优先级设置为-20；同一物理核的逻辑处理器 6 和 7 固定为 2.1 GHz。测试期间临时禁止宿主机休眠并停止桌面文件索引服务，测试结束后恢复处理器频率和休眠设置。早期准备阶段受宿主机休眠影响的样本不纳入统计，并在相同条件下重新测试。

表 2 实验环境

项目	配置
QEMU	8.2.9, x86-64 系统模式, 单线程 TCG
宿主处理器	Intel Core i7-1260P, 逻辑处理器 7（性能核），调度优先级-20
宿主频率	同核逻辑处理器 6 和 7 固定为 2.1 GHz
客户机	Debian 12, Linux 6.1.0-53-cloud-amd64, 1 个虚拟处理器
SoftTLB	动态伸缩主表加 8 项牺牲 TLB；或每个 MMU 模式固定 4096 项并关闭牺牲 TLB
客户机内存	一般负载 1 GiB, GAPBS 为 4 GiB, 微基准为 512 MiB
大页	透明大页设置为始终启用, 并检查相关负载的实际覆盖
重复次数	每个负载和方案独立运行 3 次, 报告算术平均执行时间

4.2 工作负载与指标

定向微基准在 128 MiB 区域执行 128 轮随机访问，分别使用 2 MiB 透明大页和 4 KiB 普通页；自适应负对照将 4 KiB 随机访问延长到 512 轮，并把 SoftTLB 固定为 64 项，使无大页复用时产生足够多的候选查询。应用集合包括 sysbench 随机内存访问、stress-ng TLB 失效压力、DaCapo 9.12 的 avrora[5]、GAPBS 的 PageRank 算法（图规模为 2^{20} 个顶点）[7]、嵌套 QEMU/基于内核的虚拟机（Kernel-based Virtual Machine, KVM）随机页访问，以及 SPEC CPU2006[6]的 429.mcf、471.omnetpp 和 483.xalancbmk 训练输入。

性能比较统一采用有效负载执行时间。sysbench、GAPBS 和 SPEC 负载使用客户机命令测量区间的墙钟时间；DaCapo 使用客户机报告的基准程序执行时间；嵌套虚拟化负载使用内层固定 524288 次访问的执行时间，不计外层虚拟机启动和设备初始化时间。stress-ng 采用固定 10 s 压力时间，其执行时间按实验设计基本恒定，因此只用于 MMU 不命中路径开销分析，不纳入应用加速比的平均值。每项测试独立运行 3 次，表中报告算术平均执行时间。单个负载的加速比按第 2.1 节定义计算；跨负载平均值采用归一化加速比的几何平均，以避免执行时间量级不同导致的加权偏差。

5 实验结果

5.1 定向微基准

表 3 给出固定 4096 项主 SoftTLB 并关闭牺牲 TLB 时的定向随机访存结果。每项结果为 3 次独立运行的算术平均值，执行时间按每次客户机访存的平均时间表示。透明大页场景中，LP 将平均访存时间从 306.179 ns 降至 219.209 ns，加速比为 1.397；LP+PTW 的加速比为 1.398。4 KiB 普通页场景中，PTW 将平均访存时间从 315.472 ns 降至 296.183 ns，加速比为 1.065；LP 因不存在可复用的大页转换而未取得加速。

表 3 定向随机访存微基准的平均执行时间及加速比

客户机页类型	Base/(ns/次访问)	LP/(ns/次访问；加速比)	PTW/(ns/次访问；加速比)	LP+PTW/(ns/次访问；加速比)
2 MiB 透明大页	306.179	219.209; 1.397	311.994; 0.981	219.025; 1.398
4 KiB 普通页	315.472	318.990; 0.989	296.183; 1.065	295.916; 1.066

独立机制计数结果显示，透明大页场景下，LP 将约 419.5 万次 TLB 表项填充降至约 6.6 万次；4 KiB 普通页场景下，PTW 在第 2 级页表产生约 419.5 万次命中。两种机制分别减少了预期的表项填充和页表层级访问，运行时间变化与机制计数结果一致。

5.2 应用负载

表 4 报告固定 4096 项主 SoftTLB 并关闭牺牲 TLB 时的应用结果。每个单元给出 3 次独立运行的平均执行时间及相对于 Base 的加速比。除 stress-ng 外，各负载均执行固定工作量；stress-ng 执行固定 10 s 压力时间，故不计算其跨方案加速收益。

表 4 应用负载的平均执行时间及加速比

工作负载及时间单位	Base 平均时间	LP 平均时间；加速比	PTW 平均时间；加速比	LP+PTW 平均时间；加速比
sysbench 随机访存/s	6.753	6.717; 1.005	6.726; 1.004	6.743; 1.002
stress-ng 固定时间/s	11.010	11.010; —	11.010; —	11.013; —
DaCapo avrora/ms	73921	76777; 0.963	81201; 0.910	75326; 0.981
SPEC 429.mcf/s	51.463	47.615; 1.081	50.510; 1.019	45.269; 1.137
GAPBS PageRank/s	5.530	5.563; 0.994	5.659; 0.977	5.734; 0.964
嵌套虚拟化内层负载/ms	277.679	120.496; 2.304	281.030; 0.988	123.494; 2.249
SPEC 471.omnetpp/s	658.780	630.775; 1.044	644.099; 1.023	647.662; 1.017
SPEC 483.xalancbmk/s	729.399	720.071; 1.013	734.182; 0.993	738.797; 0.987
7 个固定工作量负载的几何平均加速比	—	1.142	0.987	1.135

LP 和 LP+PTW 在 7 个固定工作量负载上的几何平均加速比分别为 1.142 和 1.135。LP 的最优结果来自嵌套虚拟化内层负载，其平均执行时间由 277.679 ms 降至 120.496 ms，加速比为 2.304；该指标仅包含内层固定次数随机访存，不包含外层虚拟机启动、设备初始化和关闭时间。429.mcf 的 LP+PTW 平均执行时间由 51.463 s 降至 45.269 s，加速比为 1.137。独立机制计数中，该负载的 LP 命中率为 68.55%，LP+PTW 中的 LP 和 PTW 命中率分别为 67.96%和 99.81%，表明它同时具有大页转换复用和非叶页表复用。

若不计具有二阶段地址转换特征的嵌套虚拟化负载，LP 和 LP+PTW 在其余 6 个固定工作量负载上的几何平均加速比分别为 1.016 和 1.013。PTW 在全部 7 个固定工作量负载上的几何平均加速比为 0.987。DaCapo 和 GAPBS 没有从新增缓存中获益，说明已经进入 MMU 不命中路径的事件仍可能缺少足够复用，缓存命中率也不能单独预测有效负载执行时间。

5.3 自适应门控与默认 SoftTLB

为检验有限地址转换覆盖和 QEMU 默认机制下的差异，表 5 给出 429.mcf 的 3 次独立重测平均值。固定容量配置使用固定 4096 项主 SoftTLB 并关闭牺牲 TLB；默认配置允许主表动态扩展，并启用 8 项牺牲 TLB。

表 5 429.mcf 在自适应方案和默认 SoftTLB 下的平均执行时间

SoftTLB 配置	方案	平均执行时间/s	加速比
固定 4096 项、无牺牲 TLB	Base	57.767	1.000
固定 4096 项、无牺牲 TLB	LP+PTW	46.967	1.230
固定 4096 项、无牺牲 TLB	A-LP+PTW	47.678	1.212
动态主表、8 项牺牲 TLB	Base	40.131	1.000
动态主表、8 项牺牲 TLB	LP	40.116	1.000
动态主表、8 项牺牲 TLB	PTW	41.730	0.962
动态主表、8 项牺牲 TLB	LP+PTW	40.665	0.987
动态主表、8 项牺牲 TLB	A-LP+PTW	40.258	0.997

固定容量配置下，LP+PTW 和 A-LP+PTW 分别取得 1.230 和 1.212 的加速比，说明自适应门控保留了该负载中的主要复用收益。默认 SoftTLB 能够在运行中扩大主表，牺牲 TLB 还能处理近期被替换的表项，因而进入本文新增缓存的请求显著减少；此时各方案均未获得明显加速，单独启用 PTW 的平均执行时间增加 3.8%。

独立机制计数结果表明，固定容量 A-LP+PTW 在约 6166 万次 LP 查询中命中 3479 万次，控制器进入 3732 个启用窗口和 253 个旁路窗口；PTW 在约 3109 万次查询中命中约 3098 万次，未进入旁路。默认配置中，牺牲 TLB 首先处理约 197 万次请求；LP 仅进入 4 个旁路窗口，PTW 未进入旁路。这表明门控根据缓存复用事件识别明显低收益阶段，但不能直接感知宿主机缓存和指令执行开销。

表 6 给出无大页复用的 4 KiB 普通页负对照。A-LP 表示仅对 LP 启用自适应收益门控。A-LP 的平均访存时间与 Base 基本相同，而始终启用 LP 使平均访存时间增加 4.5%。机制计数运行中，A-LP 旁路 738314/861993 次候选查询，即 85.7%，证明门控能够在明显缺少大页复用时停止多数无效查询。

表 6 4 KiB 普通页随机负对照的平均执行时间

方案	平均时间/(ns/次访问)	加速比
Base	349.921	1.000
LP	365.734	0.957
自适应大页缓存方案 (A-LP)	349.917	1.000

5.4 收益边界

设 Base 执行时间中可由 MMU 不命中路径缓存缩短的比例为 f ，该部分的局部加速比为 s ，则根据 Amdahl 定律，整个有效负载的理论加速比上限为

$$S_{total} = 1 / [(1 - f) + f/s] \quad (2)$$

现代 QEMU 的动态主表和牺牲 TLB 已处理大多数地址转换请求。当 TLB 表项填充次数低于访存次数的 0.5% 时，即使单次填充明显加快， f 仍可能过小。另一方面，缓存查询、4 KiB 表项安装和地址空间解析仍然存在，较高命中率也可能被这些固定成本抵消。因此，429.mcf 的收益来自较高表项填充比例及两类局部性共同作用，而 GAPBS、DaCapo 和 483.xalancbmk 的结果表明，简单增加缓存并不必然缩短有效负载执行时间。

6 讨论

本文将大页转换复用和非叶页表复用置于现代 QEMU 的 MMU 不命中路径，以不改变公共 SoftTLB 命中路径为设计约束，并显式区分失效覆盖范围与线性转换范围。与直接扩大主 SoftTLB

或增加关联度相比，该方案不会使每次客户机访存承担额外比较；与绕过 QEMU 地址空间安装过程相比，该方案保留了 MMIO、权限和观察点语义。LP 和 PTW 可以独立启用，也可由收益门控根据运行阶段选择性旁路。

固定 4096 项主 SoftTLB 并关闭牺牲 TLB 是一种用于暴露容量压力的受控配置，不代表 QEMU 默认配置，也不能直接等同于具体硬件 TLB。实验仅覆盖 x86-64 客户机、单虚拟处理器和单线程翻译代码生成器（Tiny Code Generator, TCG），宿主机也仅有一台 Intel Core i7-1260P；同一物理核的另一硬件线程仍在线，每个方案仅独立运行 3 次。因此，表中小幅时间变化只能作为初步实验结果。软件 MMU 相关 C 函数的周期占比不包含生成代码中内联的 SoftTLB 命中查询，故只是本文所定义 MMU 不命中路径开销的下界。

当前门控阈值根据一次 TLB 表项填充或所跳过的页表层级数设定，状态按虚拟处理器聚合，尚未区分地址空间、MMU 模式或程序执行阶段。该方法能够旁路明显缺少复用的区间，却不能直接观测宿主机缓存、分支预测和调度开销，因而不能保证所有有效负载均获得加速。后续需要增加重复次数和宿主机数量，隔离同核硬件线程，并在更大图分析、服务器负载和多虚拟处理器条件下调节采样窗口、收益阈值及状态粒度。

7 相关工作

Tong 等系统研究 QEMU 1.7.0 的访存模拟，比较 SoftTLB 容量、牺牲 TLB、关联度、大页和 TLB 表项填充优化[1]；Hong 等进一步分析 QEMU 2.2.0 的控制转移与内存虚拟化，强调动态容量和大页失效的共同影响[2]。本文针对已具有动态主表和牺牲 TLB 的 QEMU 8.2.9，重点处理两级表均未命中后的地址转换过程。

Barr 等提出硬件地址转换缓存，通过保存部分转换结果跳过页表层级[3]。Victima 利用末级缓存扩大地址转换覆盖范围[8]，Utopia 通过受限与灵活映射结合降低转换成本[9]，Ninja 面向嵌套地址转换提供硬件加速[10]。这些工作证明转换复用和减少页表层级的价值；本文研究相同局部性在软件全系统模拟器中的实现边界，并把软件散列计算、分支、失效和 QEMU 地址空间语义纳入设计。

HSPT 通过宿主页表和硬件 TLB 直接服务客户机访问，并以信号处理维护影子映射[12]；BTMMU 通过双 TLB、硬件异常入口和内核影子页表支持更广泛的跨指令集体系结构地址空间与页大小组合[13]。二者的公共路径不同于 QEMU SoftTLB，但影子项缺失后的 MMU 不命中路径仍需执行客户机页表遍历和映射建立。本文补充其未重点研究的路径内部缓存优化，并以固定容量实验估计硬件命中受限时的潜在收益。

8 结论

本文面向全系统动态二进制翻译器的 MMU 不命中处理，设计了大页转换缓存方案 LP、非叶页表缓存方案 PTW 及自适应收益门控。新增结构仅在主 SoftTLB 和牺牲 TLB 均未命中后工作，不增加公共命中路径的指令；缓存命中仍经过原有 4 KiB 表项安装，并通过独立的线性转换范围和统一代际计数器保持嵌套翻译及失效语义。实验表明，数据访问 TLB 表项填充次数虽然低于客户机访存次数的 0.5%，软件 MMU 相关 C 函数仍可占宿主周期的 16.84%。固定 4096 项主表并关闭牺牲 TLB 时，LP 和 PTW 在相应微基准上分别取得 1.397 和 1.065 的加速比；LP 和 LP+PTW 在 7 个固定工作量应用上的几何平均加速比分别为 1.142 和 1.135，嵌套虚拟化内层负载的最高加速比为 2.304。A-LP+PTW 在固定容量 429.mcf 上取得 1.212 的加速比，并在 4 KiB 负对照中旁路 85.7% 的候选查询。默认动态 SoftTLB 配置和部分应用尚未获得加速，说明软件查询成本仍需按负载控制。后续将把所提机制移植到以 HSPT 和 BTMMU 为代表的、采用硬件 TLB 加速的跨指令集异构虚拟

化平台，在真实硬件 TLB 命中率、硬件异常入口和影子映射维护条件下测试剩余 MMU 不命中路径的优化效果，并与纯软件 QEMU 结果进行比较。届时将以固定工作量的内层或有效负载三次独立运行的平均执行时间作为主要指标，重点分析硬件加速公共命中路径后，两类缓存对最终应用性能贡献。

参考文献

- [1] TONG X, KOJU T, KAWAHITO M, et al. Optimizing memory translation emulation in full system emulators[J]. ACM Transactions on Architecture and Code Optimization, 2015, 11(4): 1-24.
- [2] HONG D Y, HSU C C, CHOU C Y, et al. Optimizing control transfer and memory virtualization in full system emulators[J]. ACM Transactions on Architecture and Code Optimization, 2015, 12(4): 1-24.
- [3] BARR T W, COX A L, RIXNER S. Translation caching: skip, don't walk the page table[C]//Proceedings of the 37th Annual International Symposium on Computer Architecture. Saint-Malo: ACM, 2010: 48-59.
- [4] BELLARD F. QEMU, a fast and portable dynamic translator[C]//Proceedings of the USENIX Annual Technical Conference. Anaheim: USENIX Association, 2005: 41-46.
- [5] BLACKBURN S M, GARNER R, HOFFMANN C, et al. The DaCapo benchmarks: Java benchmarking development and analysis[C]//Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications. Portland: ACM, 2006: 169-190.
- [6] HENNING J L. SPEC CPU2006 benchmark descriptions[J]. ACM SIGARCH Computer Architecture News, 2006, 34(4): 1-17.
- [7] BEAMER S, ASANOVIC K, PATTERSON D. The GAP benchmark suite[EB/OL]. (2015)[2026-09-12]. <https://github.com/sbeamer/gapbs>.
- [8] KANELLOPOULOS K, NAM H C, BOSTANCI N, et al. Victima: drastically increasing address translation reach by leveraging underutilized cache resources[C]//Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. Toronto: ACM, 2023: 1178-1195.
- [9] KANELLOPOULOS K, BERA R, STOJILJKOVIC K, et al. Utopia: fast and efficient address translation via hybrid restrictive and flexible virtual-to-physical address mappings[C]//Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. Toronto: ACM, 2023: 1196-1212.
- [10] ZHAO L Y, WANG Z W, LIU F X, et al. Ninja: a hardware assisted system for accelerating nested address translation[C]//Proceedings of the 42nd IEEE International Conference on Computer Design. Milan: IEEE, 2024: 426-433.
- [11] LINDSAY N, BHATTACHARJEE A. Understanding address translation scaling behaviours using hardware performance counters[C]//Proceedings of the IEEE International Symposium on Workload Characterization. Vancouver: IEEE, 2024: 236-246.
- [12] WANG Z, LI J J, WU C G, et al. HSPT: practical implementation and efficient management of embedded shadow page tables for cross-ISA system virtual machines[C]//Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Istanbul: ACM, 2015: 53-64.
- [13] HUANG K, ZHANG F X, LI C, et al. BTMMU: an efficient and versatile cross-ISA memory virtualization[C]//Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Virtual Event, USA: ACM, 2021: 71-83.